

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

ENTREGA FINAL — PROYECTO INTEGRADOR BASES DE DATOS 2

JOHAN SEBASTIAN BONILLA GAMEZ
JUAN SEBASTIAN MORALES HERRERA
SANTIAGO YESID TIUSABA ARANDA

FRANCISCO RAFAEL BARGUIL VANEGAS

UNIVERSIDAD EL BOSQUE

2026

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

Contenido

1.	IDENTIDAD DEL FESTIVAL	3
1.1.	Información General.....	3
1.2.	Escenarios.....	3
1.3.	Tipos de Boleta.....	3
1.4.	Descripción del Festival	4
2.	ARQUITECTURA POLÍGLOTA DEL SISTEMA.....	4
3.	AVANCE 1 - NÚCLEO TRANSACCIONAL POSTGRESQL	5
3.1.	Correcciones incorporadas tras retroalimentación	5
3.2.	Modelo Entidad-Relación.....	6
3.3.	Modelo Relacional Normalizado en 3FN.....	7
3.4.	Scripts SQL.....	10
3.5.	Triggers.....	11
3.6.	Procedimiento Almacenado.....	13
3.7.	Consultas SQL.....	14
4.	AVANCE 2 - CAPA DOCUMENTAL MONGODB	15
4.1.	Correcciones incorporadas tras retroalimentación	15
4.2.	Justificación del uso de MongoDB.....	16
4.3.	Diseño de las 4 colecciones	16
4.4.	Scripts MongoDB.....	20
4.5.	Consultas MongoDB	21
5.	ENTREGA FINAL - INTEGRACIÓN POLÍGLOTA	30
5.1.	Script de integración.....	30
5.2.	Caso 1 - Reporte de Artista.....	30
5.3.	Caso 2 - Panel de Incidentes por Escenario.....	31
6.	JUSTIFICACIÓN TEÓRICA — ACID, BASE Y CAP.....	33
6.1.	Propiedades ACID en PostgreSQL.....	33
7.	REPOSITORIO GITHUB	36

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

1. IDENTIDAD DEL FESTIVAL

1.1. Información General

Campo	Detalle
Nombre del festival	Festival Soundwave Colombia
Género musical principal	Electrónica / EDM
Géneros secundarios	Indie, Rock alternativo
Ciudad	Bogotá, Colombia
Sede	Parque Metropolitano Simón Bolívar
Duración	3 días: 14, 15 y 16 de noviembre de 2026
Número de escenarios	4 escenarios
Capacidad total diaria	40.000 personas
Número de artistas	30 artistas en total

1.2. Escenarios

Escenario	Tipo	Capacidad
Mainstage	Principal	25.000 personas
Nebula	Alternativo	8.000 personas
Roots	Acústico	4.000 personas
Underground	Alternativo	3.000 personas

1.3. Tipos de Boleta

Tipo	Precio	Cupo máximo	Beneficios
Early Bird	\$100.000	5.000	Acceso general anticipado
General	\$150.000	20.000	Acceso a todos los escenarios
VIP	\$350.000	8.000	Zona exclusiva + acceso preferencial

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

Palco	\$600.000	2.000	Palco exclusivo + meet & greet
-------	-----------	-------	--------------------------------

1.4. Descripción del Festival

El Festival Soundwave Colombia es un evento anual de música electrónica e indie realizado en Bogotá, este festival cuenta con 4 escenarios simultáneos con programación diferenciada por género, acceso segmentado por zonas según el tipo de boleta adquirida, zona VIP exclusiva con meet & greet con artistas seleccionados y staff especializado por escenario en las áreas de seguridad, logística, sonido y atención médica.

2. ARQUITECTURA POLÍGLOTA DEL SISTEMA

El sistema implementa una arquitectura políglota que combina dos motores de base de datos con roles complementarios:

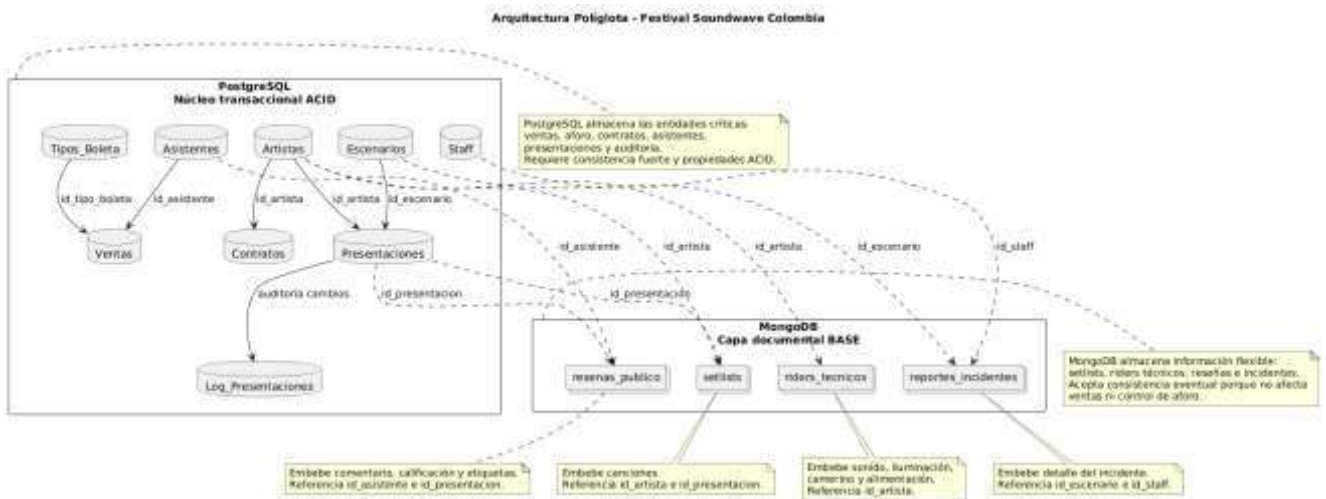
Motor	Plataforma	Rol	Modelo
PostgreSQL	Neon	Núcleo transaccional	ACID — CP
MongoDB	Atlas	Capa documental	BASE — AP

PostgreSQL: Almacena todas las entidades que requieren consistencia estricta, integridad referencial y operaciones transaccionales: ventas, contratos, presentaciones, boletas.

MongoDB: Almacena toda la información documental con estructura variable y alto volumen: setlists, riders técnicos, reseñas del público, reportes de incidentes.

El script de integración en Python: Conecta ambos motores en tiempo real y presenta los datos combinados en una interfaz gráfica unificada.

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	



3. AVANCE 1 - NÚCLEO TRANSACCIONAL POSTGRESQL

3.1. Correcciones incorporadas tras retroalimentación

Posterior a la revisión y calificación del Avance 1, se aplicaron las siguientes correcciones:

Corrección 1 - Restricción UNIQUE en Presentaciones

Se agregó una restricción UNIQUE sobre los campos id_artista, fecha y hora_inicio para evitar que un artista pueda estar programado en dos escenarios al mismo tiempo:

```
ALTER TABLE Presentaciones
```

```
ADD CONSTRAINT uq_artista_fecha_hora
```

```
UNIQUE (id_artista, fecha, hora_inicio);
```

Corrección 2 - Trigger de aforo con FOR UPDATE

El trigger trg_validar_aforo fue corregido para incluir la cláusula FOR UPDATE en la consulta de cupo máximo. Esto bloquea la fila del tipo de boleta durante la verificación, eliminando la posible condición de carrera cuando dos compras llegan simultáneamente al sistema.

Corrección 3 - Procedimiento almacenado corregido

El procedimiento sp_reporte_ventas fue corregido para eliminar la inconsistencia entre su nombre y su implementación. La versión corregida agrupa por tipo de boleta y fecha, alineando el nombre con lo que el código realmente hace.

Corrección 4 - Consulta 1 corregida

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

La consulta de ranking de artistas tenía un JOIN cíclico inválido JOIN Tipos_Boleta tb ON tb.id_tipo = tb.id_tipo que comparaba la tabla consigo misma generando un producto cartesiano. Todos los artistas mostraban exactamente 30 ventas y 61 boletas, lo cual era incorrecto. La consulta fue reescrita para eliminar el JOIN cíclico.

Corrección 5 - Consulta 2 corregida

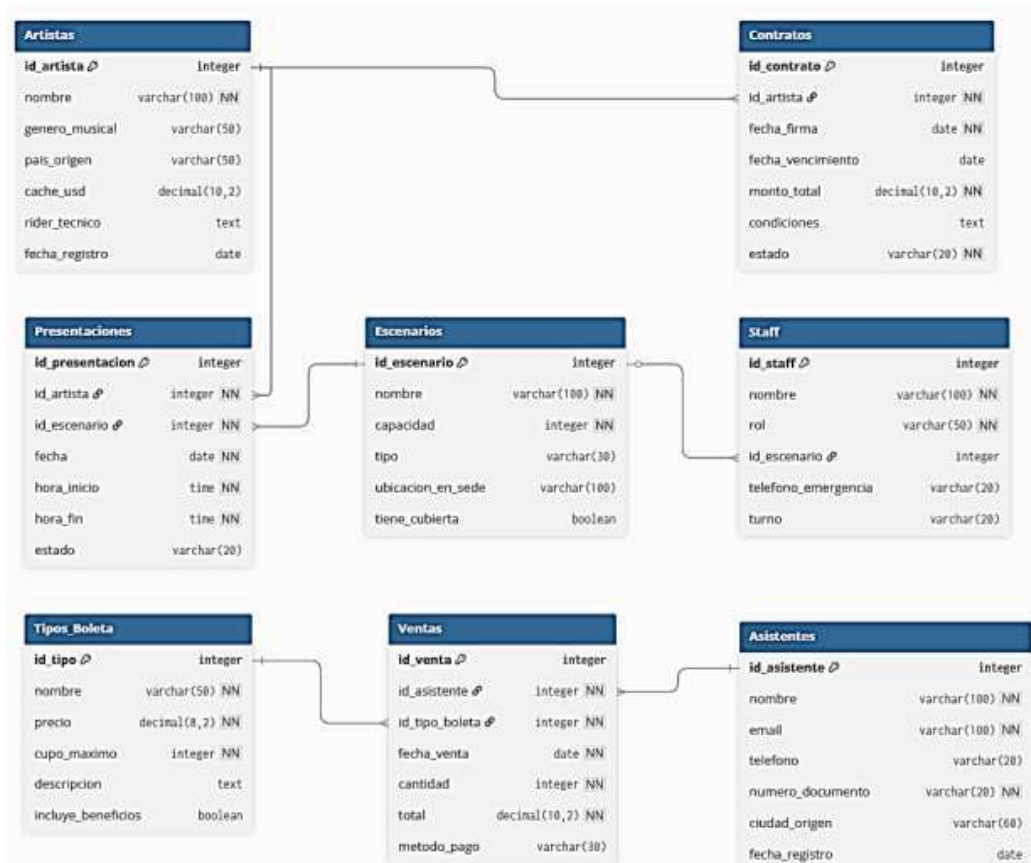
La consulta de ocupación tenía inconsistencia entre su título y su implementación. El título decía ocupación por escenario, pero el código agrupaba por tipo de boleta. Se alineó el nombre con lo que el código realmente hace.

3.2. Modelo Entidad-Relación

El núcleo transaccional está compuesto por 8 entidades principales con las siguientes relaciones:

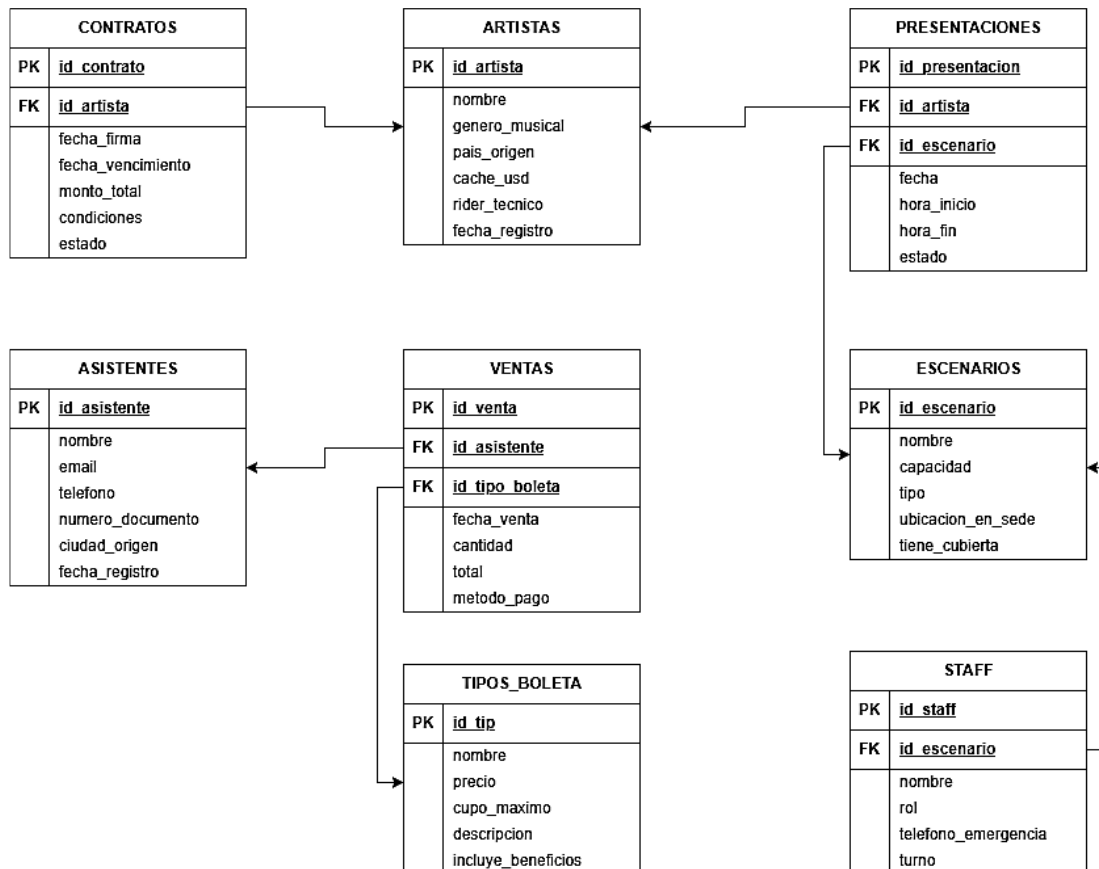
Relación	Tipo	Descripción
Artistas - Contratos	1:N	Un artista puede tener uno o más contratos
Artistas - Presentaciones	1:N	Un artista puede tener múltiples presentaciones
Escenarios -Presentaciones	1:N	Un escenario alberga múltiples presentaciones
Escenarios - Staff	1:N	Un escenario tiene múltiples miembros de staff
Asistentes - Ventas	1:N	Un asistente puede realizar múltiples compras
Tipos_Boleta - Ventas	1:N	Un tipo de boleta aparece en múltiples ventas

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	



3.3. Modelo Relacional Normalizado en 3FN

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	



Modelo relacional final:

- ARTISTAS (id_artista PK, nombre, genero_musical, pais_origen, cache_usd, rider_tecnico, fecha_registro)
- CONTRATOS (id_contrato PK, id_artista FK→ARTISTAS, fecha_firma, fecha_vencimiento, monto_total, condiciones, estado)
- ESCENARIOS (id_escenario PK, nombre, capacidad, tipo, ubicacion_en_sede, tiene_cubierta)
- PRESENTACIONES (id_presentacion PK, id_artista FK→ARTISTAS, id_escenario FK→ESCENARIOS, fecha, hora_inicio, hora_fin, estado)
- TIPOS_BOLETA (id_tipo PK, nombre, precio, cupo_maximo, descripcion, incluye_beneficios)
- ASISTENTES (id_asistente PK, nombre, email UNIQUE, telefono, numero_documento UNIQUE, ciudad_origen, fecha_registro)
- VENTAS (id_venta PK, id_asistente FK→ASISTENTES, id_tipo_boleta FK→TIPOS_BOLETA, fecha_venta, cantidad, total, metodo_pago)

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

- STAFF (id_staff PK, nombre, rol, id_escenario FK→ESCENARIOS, telefono_emergencia, turno)
- LOG_PRESENTACIONES (id_log PK, id_presentacion, campo_modificado, valor_anterior, valor_nuevo, fecha_cambio, usuario)

Justificación de normalización:

Primera Forma Normal (1FN): Todos los atributos son atómicos e indivisibles. No existen grupos repetidos ni atributos multivaluados. Cada venta es una fila independiente, el género musical registra únicamente el género principal y las condiciones del contrato se almacenan como texto único.

Segunda Forma Normal (2FN): Todas las tablas utilizan llaves primarias simples de un solo atributo, por lo tanto, es imposible que existan dependencias parciales. El modelo cumple 2FN por diseño.

Tercera Forma Normal (3FN): Se identificaron y eliminaron todas las dependencias transitivas:

Dependencia detectada	Solución aplicada
Precio de boleta dependía de id_tipo_boleta en Ventas	Precio vive únicamente en Tipos_Boleta
Capacidad del escenario dependía de id_escenario en Presentaciones	Capacidad vive únicamente en Escenarios
Nombre y email del asistente dependían de id_asistente en Ventas	Esos datos viven únicamente en Asistentes
Nombre del escenario dependía de id_escenario en Staff	Nombre vive únicamente en Escenarios
Datos del artista dependían de id_artista en Contratos	Datos del artista viven únicamente en Artistas

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

3.4. Scripts SQL

Script 1 - creacion_tablas.sql

Crea las 9 tablas del sistema con sus restricciones completas. Incluye llaves primarias con SERIAL, llaves foráneas con ON DELETE RESTRICT y ON UPDATE CASCADE, restricciones CHECK en todos los campos críticos, UNIQUE en email y numero_documento de Asistentes, UNIQUE compuesto en Presentaciones para evitar artista en dos escenarios simultáneamente, CHECK de fechas válidas del festival y 8 índices en campos de búsqueda frecuente.

```

1 -- Script: creacion_tablas.sql
2
3 CREATE TABLE Artistas (
4     id_artista SERIAL PRIMARY KEY,
5     nombre VARCHAR(100) NOT NULL,
6     genero_musical VARCHAR(50) NOT NULL,
7     pais_origen VARCHAR(50) NOT NULL,
8     cache_usd DECIMAL(10,2) CHECK (cache_usd > 0),
9     rider_tecnico TEXT,
10    fecha_registro DATE DEFAULT CURRENT_DATE
11 );
12
13 CREATE TABLE Contratos (
14     id_contrato SERIAL PRIMARY KEY,
15     id_artista INTEGER NOT NULL,
16     fecha_firma DATE NOT NULL,
17     fecha_vencimiento DATE,
18     monto_total DECIMAL(10,2) NOT NULL CHECK (monto_total > 0),
19 );

```

Connected (17 queries)

Run Explain Analyze 244ms No result

1: CREATE 2: CREATE 3: CREATE 4: CREATE 5: CREATE 6: CREATE 7: CREATE 8: CREATE 9: CREATE 10: CREATE 11: CREATE 12: CREATE 13: CREATE 14: CREATE 15: CREATE 16: CREATE 17: CREATE

Statement executed successfully

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

Script 2 - datos_prueba.sql

Inserta los datos de prueba con un mínimo de 30 registros por entidad principal: 30 artistas nacionales e internacionales, 30 contratos, 30 presentaciones distribuidas en los 3 días y 4 escenarios, 30 asistentes de diferentes ciudades de Colombia, 30 ventas con diferentes métodos de pago, 4 escenarios, 4 tipos de boleta y 15 miembros de staff.

```

209 -- VERIFICACION
210
211 SELECT 'Artistas' AS tabla, COUNT(*) AS registros FROM Artistas
212 UNION ALL
213 SELECT 'Asistentes', COUNT(*) FROM Asistentes
214 UNION ALL
215 SELECT 'Ventas', COUNT(*) FROM Ventas
216 UNION ALL
217 SELECT 'Presentaciones', COUNT(*) FROM Presentaciones
218 UNION ALL
219 SELECT 'Contratos', COUNT(*) FROM Contratos
220 UNION ALL
221 SELECT 'Escenarios', COUNT(*) FROM Escenarios
222 UNION ALL
223 SELECT 'Tipos_Boleta', COUNT(*) FROM Tipos_Boleta
224 UNION ALL
225 SELECT 'Staff', COUNT(*) FROM Staff;

```

Connected (9 queries) ⚠ This query will still run OK, but the last 5006 characters will be truncated from query history

Run Explain Analyze

1: INSERT 4 2: INSERT 4 3: INSERT 30 4: INSERT 30 5: INSERT 30 6: INSERT 30 7: INSERT 30 8: INSERT 15 9: SELECT 8

#	tabla	registros
1	Artistas	30
2	Asistentes	30
3	Ventas	30
4	Presentaciones	30
5	Contratos	30
6	Escenarios	4
7	Tipos_Boleta	4
8	Staff	15

3.5. Triggers

Trigger 1 - Validación de aforo

Nombre: trg_validar_aforo Evento: BEFORE INSERT en la tabla Ventas Función: fn_validar_aforo()

Antes de registrar una venta, verifica que la suma de boletas vendidas de ese tipo más las nuevas solicitadas no supere el cupo máximo definido en Tipos_Boleta. Utiliza FOR UPDATE para bloquear la fila durante la verificación y evitar condiciones de carrera en compras simultáneas. Si el cupo está agotado lanza un RAISE EXCEPTION con el detalle del error. Es crítico para un festival con capacidad limitada por tipo de boleta. Garantiza que

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

nunca se vendan más boletas de las permitidas independientemente de cuántas compras simultáneas lleguen al sistema.

```

1  -- PRUEBA 1 - TRIGGER 1
2  INSERT INTO Ventas (id_asistente, id_tipo_boleta, fecha_venta, cantidad, total, metodo_pago)
3  VALUES (1, 1, '2026-05-20', 25000, 2500000.00, 'tarjeta');

```

Connected (1 query)

Run Explain Analyze

ERROR: Cupo agotado para este tipo de boleta. Vendidas: 15, Cupo maximo: 5000, Solicitadas: 25000 (SQLSTATE P0001)

Trigger 2 - Auditoría de presentaciones

Nombre: trg_auditoria_presentaciones Evento: AFTER UPDATE en la tabla Presentaciones

Función: fn_auditoria_presentaciones()

Cada vez que se modifica una presentación registra automáticamente en Log_Presentaciones el campo modificado, el valor anterior, el valor nuevo, la fecha del cambio y el usuario que lo realizó. Audita cambios en: fecha, hora_inicio, hora_fin, estado e id_escenario. Utiliza IS DISTINCT FROM para comparación segura con valores NULL.

Pertinencia: En un festival con programación en vivo es fundamental tener trazabilidad de cualquier cambio de última hora en la agenda de artistas.

```

1  -- PRUEBA - TRIGGER 2
2
3  UPDATE Presentaciones
4  SET estado = 'cancelada'
5  WHERE id_presentacion = 1;

```

Connected (1 query)

Run Explain Analyze

Statement executed successfully

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

<pre>-- PRUEBA - TRIGGER 2 1 SELECT * FROM Log_Presentaciones;</pre>																							
Connected (1 query)																							
Run Explain Analyze																							
<table> <tr> <th>#</th><th>id_log</th><th>id_presentacion</th><th>campo_modificado</th><th>valor_anterior</th><th>valor_nuevo</th><th>fecha_cambio</th><th>usuario</th></tr> <tr> <td>1</td><td>1</td><td>1</td><td>estado</td><td>programa</td><td>cancelada</td><td>2026-05-08 20:55:35.463457</td><td>usuario_senar</td></tr> </table>								#	id_log	id_presentacion	campo_modificado	valor_anterior	valor_nuevo	fecha_cambio	usuario	1	1	1	estado	programa	cancelada	2026-05-08 20:55:35.463457	usuario_senar
#	id_log	id_presentacion	campo_modificado	valor_anterior	valor_nuevo	fecha_cambio	usuario																
1	1	1	estado	programa	cancelada	2026-05-08 20:55:35.463457	usuario_senar																

3.6.Procedimiento Almacenado

sp_reporte_ventas(fecha_consulta DATE)

Recibe una fecha como parámetro y retorna una tabla con el tipo de boleta, total de boletas vendidas y total de ingresos para esa fecha. Agrupa las ventas por tipo de boleta y las ordena por ingresos de mayor a menor.

Pertinencia: Permite a la organización del festival monitorear el comportamiento de ventas por día y tomar decisiones operativas sobre logística y programación.

```

1
2
3 -- PROCEDIMIENTO
4
5 CREATE OR REPLACE FUNCTION sp_reporte_ventas(fecha_consulta DATE)
6 RETURNS TABLE (
7     escenario      VARCHAR,
8     tipo_boleta    VARCHAR,
9     total_boletas  BIGINT,
10    total_ingresos  NUMERIC
11 ) AS $$
12 BEGIN
13     RETURN QUERY
14     SELECT
15         e.nombre::VARCHAR      AS escenario,
16         tb.nombre::VARCHAR     AS tipo_boleta,
17         SUM(v.cantidad)        AS total_boletas,
18         SUM(v.total)          AS total_ingresos
19     FROM Ventas v
20     JOIN Tipos_Boleta tb
21         ON v.id_tipo_boleta = tb.id_tipo
22     JOIN Asistentes a
23         ON v.id_asistente = a.id_asistente
24     CROSS JOIN Escenarios e
25     JOIN Presentaciones p
26         ON p.id_escenario = e.id_escenario
27         AND p.fecha = fecha_consulta
28     WHERE v.fecha_venta = fecha_consulta
29     GROUP BY e.nombre, tb.nombre
30     ORDER BY e.nombre, total_ingresos DESC;
31 END;
32 $$ LANGUAGE plpgsql;

```

Connected (1 query)

Run Explain Analyze

Statement executed successfully

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

3.7. Consultas SQL

Consulta 1 - Top artistas por ingresos generados

Técnica: JOIN múltiple + SUM + ORDER BY + LIMIT Propósito: Identificar qué artistas generan mayor retorno económico para el festival, mostrando total de ventas, boletas vendidas e ingresos generados por artista.

```

1 -- Script: consultas.sql
2
3 -- CONSULTA 1
4
5 SELECT
6   a.nombre,                AS artista,
7   a.pais_origen,
8   a.genero_musical,
9   COALESCE(SUM(v.cantidad), 0) AS total_ventas,
10  COALESCE(SUM(v.cantidad), 0) AS boletas_vendidas,
11  COALESCE(SUM(v.total), 0) AS ingresos_generados
12 FROM Artistas a
13 JOIN Presentaciones p
14   ON a.id_artista = p.id_artista
15 JOIN TiposBoleta tb

```

#	artista	pais_origen	genero_musical	TOTAL_ventas	boletas_vendidas	ingresos_generados
1	Turist	Perú	Electrónica	10	10	1200000.00
2	Chocallatzen	Colombia	Reggaeton	10	10	1200000.00
3	Don Nephew	Perú	Electrónica	10	10	1200000.00
4	Micap	Perú	Electrónica	10	10	1200000.00
5	Turismo	Perú	Trova	10	10	1200000.00
6	Marta María	Perú	Pop	10	10	1200000.00
7	Mercedes Brothers	Colombia	Reggaeton	10	10	1200000.00
8	Nosifer Period	Colombia	Reggaeton	10	10	1200000.00
9	Floating Point	Perú	Electrónica	10	10	1200000.00
10	Charlita de Mito	Perú	Trova	10	10	1200000.00

Consulta 2 - Ocupación por tipo de boleta

Técnica: LEFT JOIN + SUM + cálculo porcentual + ROUND Propósito: Monitorear qué tipos de boleta están cerca de su límite de cupo, mostrando boletas vendidas, disponibles y porcentaje de ocupación.

```

23 -- Script: consultas.sql
24
25 -- CONSULTA 2
26
27 SELECT
28   tb.nombre,                AS tipo_boleta,
29   tb.precio,
30   tb.cupo_maximo,
31   COALESCE(SUM(v.cantidad), 0) AS boletas_vendidas,
32   COALESCE(SUM(v.cantidad), 0) AS boletas_disponibles,
33   ROUND(
34     (COALESCE(SUM(v.cantidad), 0) * 100.0)
35     / tb.cupo_maximo, 2)
36   AS porcentaje_ocupacion
37 FROM TiposBoleta tb
38 LEFT JOIN Ventas v
39   ON tb.id_tipo_boleta = v.id_tipo_boleta

```

#	tipo_boleta	precio	cupos_maximos	boletas_vendidas	boletas_disponibles	porcentaje_ocupacion
1	Pequeño	10000.00	1000	1	999	0.10
2	Early Bird	15000.00	1000	10	990	0.10
3	VIP	20000.00	1000	10	990	0.10
4	General	15000.00	10000	10	9990	0.10

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

Consulta 3 - Asistentes con mayor gasto total

Técnica: JOIN + GROUP BY + HAVING + COUNT DISTINCT + ORDER BY Propósito: Detectar los asistentes de mayor valor económico para el festival, mostrando número de compras, tipos de boleta distintos adquiridos y gasto total.

```

42 -- CONSULTA 3
43
44 SELECT
45   a.nombre,                AS asistente,
46   a.ciudad_origen,
47   a.email,
48   COUNT(v.id_venta)        AS numero_compras,
49   COUNT(DISTINCT v.id_tipo_boleta) AS tipos_boleta_distintos,
50   SUM(v.cantidad)          AS total_boletas,
51   SUM(v.total)             AS gasto_total
52 FROM asistentes a
53 JOIN ventas v
54   ON a.id_asistente = v.id_asistente
55 GROUP BY a.id_asistente, a.nombre, a.ciudad_origen, a.email
56 HAVING COUNT(v.id_venta) >= 1
57 ORDER BY gasto_total DESC

```

#	asistente	ciudad_origen	email	numero_compras	tipos_boleta_distintos	total_boletas	gasto_total
1	Sebastian Gutierrez	Carigona	sebastian.gutierrez@gmail.com	3	1	2	370000.00
2	Carlos Ramirez	Reguá	carlos.ramirez@gmail.com	2	1	3	70000.00
3	Walter Wain	Armenia	walter.wain@gmail.com	3	1	2	60000.00
4	Felipe Vargas	Neiva	felipe.vargas@gmail.com	1	1	1	20000.00
5	Andrés Torres	Cali	andres.torres@gmail.com	1	1	1	10000.00
6	Andrés Ospina	Pasta	andres.ospina@gmail.com	1	1	2	30000.00
7	Laura Jarama	Reguá	laura.jarama@gmail.com	1	1	1	10000.00
8	Paula Alvarado	Reguá	paula.alvarado@gmail.com	1	1	1	10000.00
9	Santiago Herrera	Barranquilla	santiago.herrera@gmail.com	1	1	1	10000.00
10	Alejandra Oroz	Tagua	alejandra.oro@gmail.com	1	1	1	10000.00

4. AVANCE 2 - CAPA DOCUMENTAL MONGODB

4.1. Correcciones incorporadas tras retroalimentación

Corrección 1 - Datos de prueba variados y realistas

Los datos de prueba originales fueron generados con bucles for y Math.random() con arreglos fijos, lo que resultaba en riders idénticos y comentarios repetidos. Se reescribió completamente el script datos_prueba.js con datos únicos y realistas para cada documento: cada artista tiene su propia consola de sonido, bebidas, snacks y requerimientos específicos, y cada reseña tiene un comentario único y detallado sobre la presentación.

Corrección 2 - Consultas más elaboradas

Las consultas originales eran find() simples. Se mejoraron las 3 consultas: la consulta 1 ahora usa proyección y sort, la consulta 2 usa operadores \$and, \$in y \$gte combinados, y la consulta 3 es un aggregation pipeline completo con 6 etapas incluyendo \$lookup para cruzar colecciones.

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

Corrección 3 - Sección BASE sin estructura repetitiva

La sección de justificación BASE original repetía la misma plantilla argumental para cada colección. Se reescribió como un argumento único y cohesionado que explica la decisión de arquitectura de forma integral.

4.2. Justificación del uso de MongoDB

El Festival Soundwave Colombia utiliza MongoDB para la capa documental por tres razones concretas:

Los documentos tienen estructura variable entre artistas. Un artista de EDM tiene requerimientos técnicos completamente distintos a uno acústico, lo que hace imposible modelarlo en columnas fijas de SQL sin desperdiciar espacio o crear columnas nulas.

El volumen de reseñas y reportes es alto y no requiere consistencia transaccional. Si una reseña llega con un segundo de retraso no afecta ninguna operación crítica del festival.

Los arreglos embebidos como las canciones de un setlist se modelan naturalmente en documentos JSON y serían ineficientes en tablas relacionales, requiriendo una tabla separada con muchas filas y JOINS costosos.

4.3. Diseño de las 4 colecciones

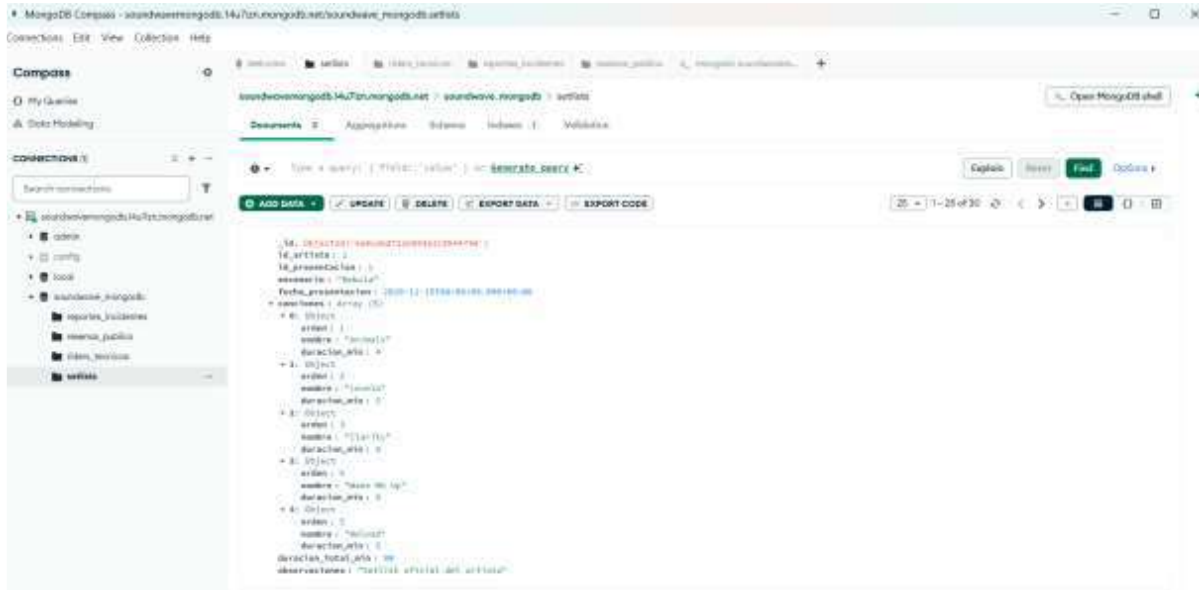
Colección setlists

Descripción: Almacena la lista de canciones que cada artista planea interpretar en su presentación.

Campo	Decisión	Justificación
canciones (array)	Embebido	Las canciones no tienen existencia propia fuera del setlist. Siempre se consultan junto al setlist completo
id_artista	Referenciado	El artista vive en PostgreSQL con todos sus datos transaccionales
id_presentacion	Referenciado	La presentación vive en PostgreSQL con restricciones transaccionales

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

Índices creados: id_artista, id_presentacion, escenario



Colección riders_tecnicos

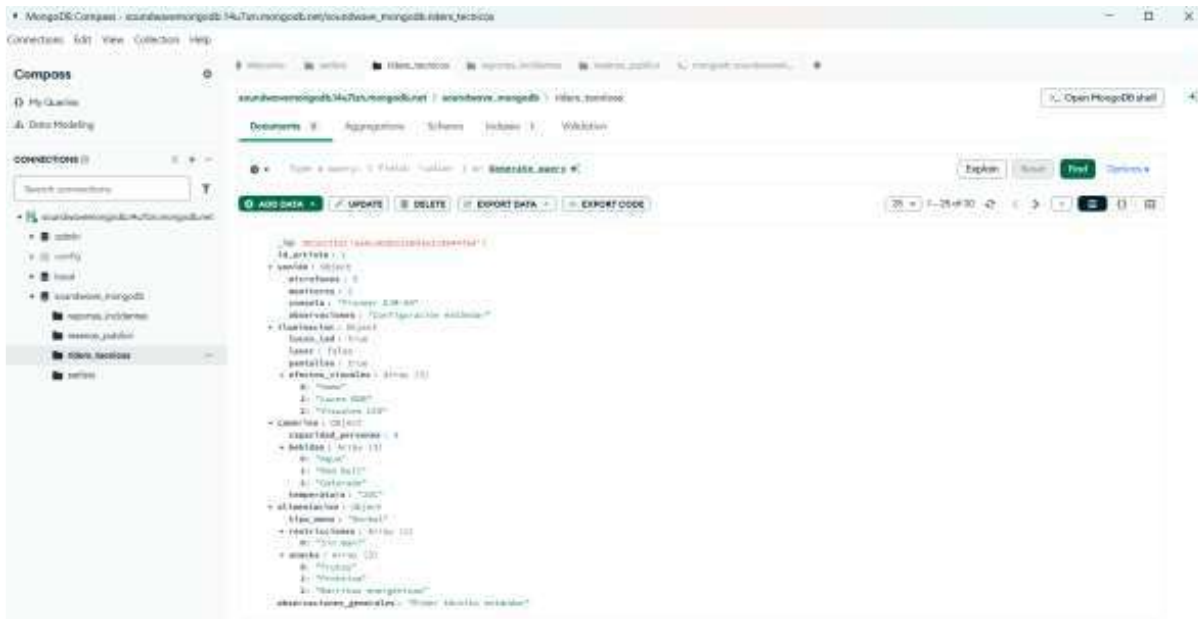
Descripción: Almacena los requerimientos técnicos detallados de cada artista organizados por categoría: sonido, iluminación, camerino y alimentación.

Campo	Decisión	Justificación
sonido, iluminacion, camerino, alimentacion	Embebido	Son subdocumentos que pertenecen exclusivamente al rider. Nunca se consultan de forma independiente
id_artista	Referenciado	El artista vive en PostgreSQL con sus datos transaccionales

Justificación específica: Cada artista tiene requerimientos completamente distintos. Martin Garrix necesita un sistema L-Acoustics K2 mientras que un artista acústico necesita monitores simples. Esta variabilidad hace imposible modelarlo en columnas fijas de SQL.

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

Índices creados: id_artista



Colección resenas_publico

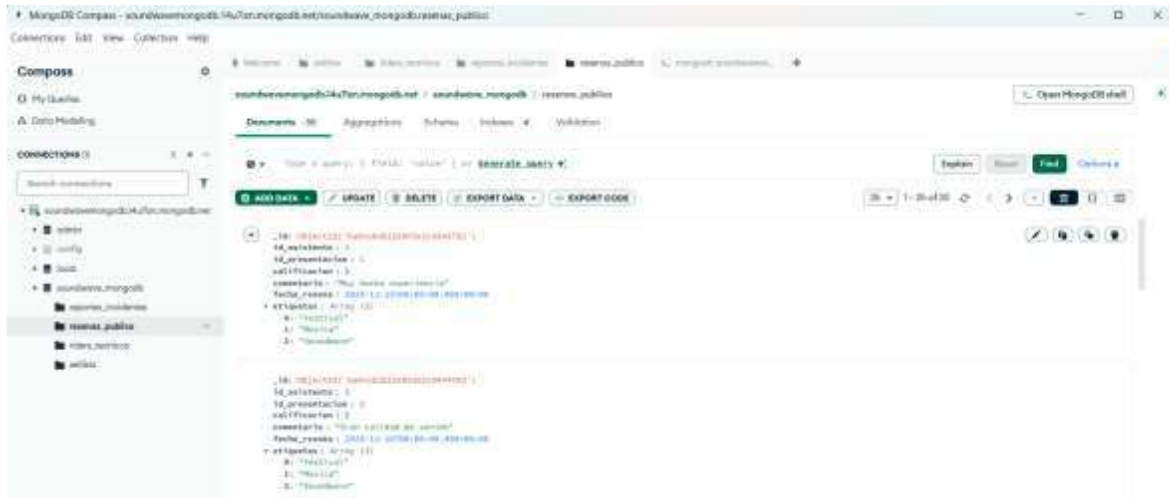
Descripción: Almacena las opiniones y calificaciones que los asistentes dejan sobre las presentaciones del festival.

Campo	Decisión	Justificación
calificacion, comentario, etiquetas	Embebido	Son parte inseparable de la reseña. No tienen sentido fuera de ella
id_asistente	Referenciado	El asistente vive en PostgreSQL con sus datos de compra
id_presentacion	Referenciado	La presentación vive en PostgreSQL con sus datos transaccionales

Justificación específica: El volumen de reseñas puede ser muy alto durante el festival. No requiere consistencia transaccional: si una reseña llega con un segundo de retraso no afecta ninguna operación crítica.

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

Índices creados: id_asistente, id_presentacion, calificacion (descendente)



Colección reportes_incidentes

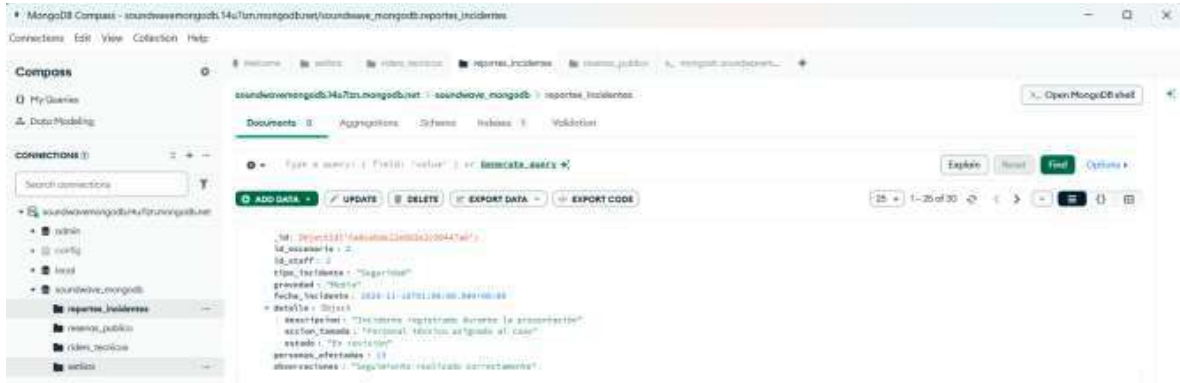
Descripción: Almacena los incidentes reportados durante el festival: problemas de sonido, altercados de seguridad, emergencias médicas y fallas lógicas.

Campo	Decisión	Justificación
detalle (descripcion, accion_tomada, estado)	Embebido	El detalle del incidente es inseparable del reporte. Siempre se consulta junto
id_escenario	Referenciado	El escenario vive en PostgreSQL con su capacidad y datos operativos
id_staff	Referenciado	El miembro de staff vive en PostgreSQL con su rol y turno asignado

Justificación específica: Cada tipo de incidente tiene campos distintos. Un incidente médico requiere número de personas afectadas. Un problema de sonido requiere descripción técnica del equipo. Esta variabilidad de esquema es el caso de uso ideal para MongoDB.

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

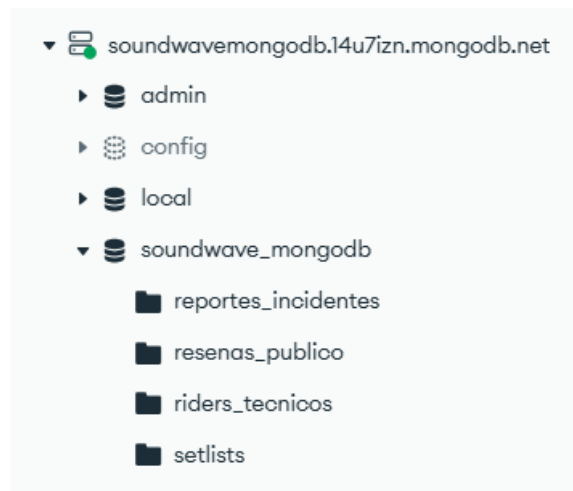
Índices creados: id_escenario, id_staff, gravedad, tipo_incidente



4.4. Scripts MongoDB

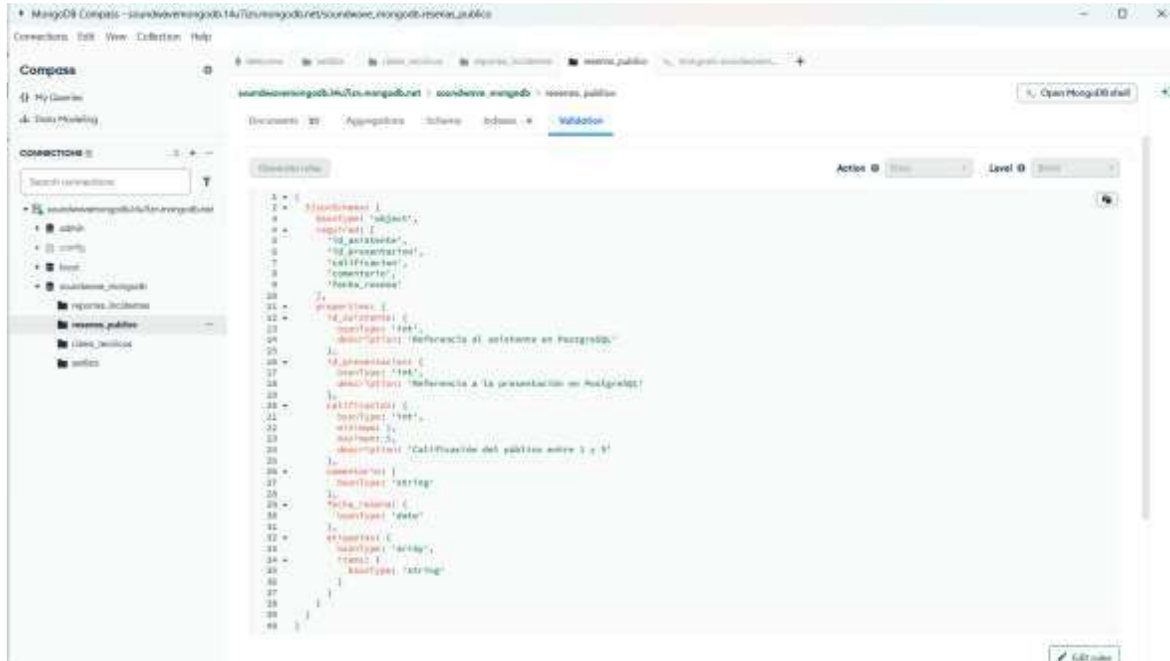
Script	Descripción
creacion_coleccion.js	Crea las 4 colecciones con validación JSON Schema e índices
datos_prueba.js	Inserta 30 documentos por colección con datos variados y realistas
consultas.js	Las 3 consultas significativas incluyendo aggregation pipeline

CAPTURA - COLECCIONES CREADAS



	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

CAPTURA - VALIDACIÓN / SCHEMA



CAPTURA - COUNT DOCUMENTS

```

> db.resenas_publico.countDocuments()
< 30

> db.reportes_incidentes.countDocuments()
< 30

> db.riders_tecnicos.countDocuments()
< 30

> db.setlists.countDocuments()
< 30

```

4.5. Consultas MongoDB

Consulta 1 — Setlists ordenados por duración

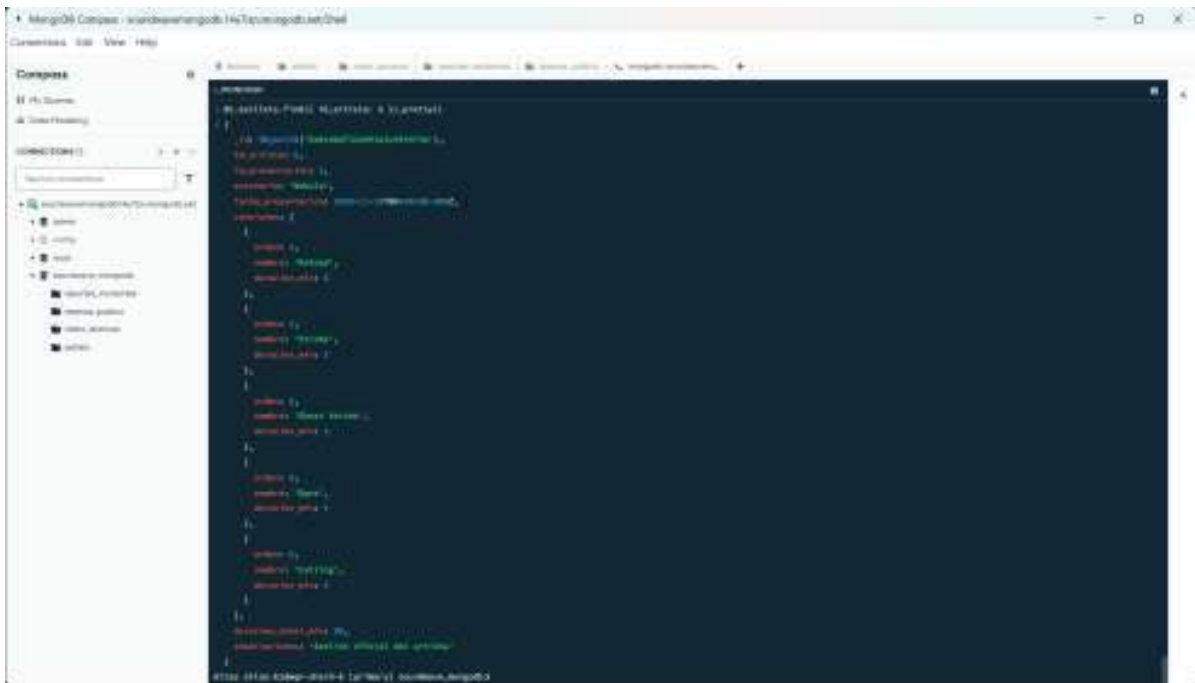
Técnica: find() con proyección de campos específicos y sort descendente por duración total.
 Propósito: Obtener la programación ordenada por duración para planificar la logística del festival.

 UNIVERSIDAD EL BOSQUE	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

```

db.setlists.find({}, {
  id_artista: 1, escenario: 1,
  fecha_presentacion: 1,
  duracion_total_min: 1,
  canciones: { $slice: 1 }
}).sort({ duracion_total_min: -1 });

```

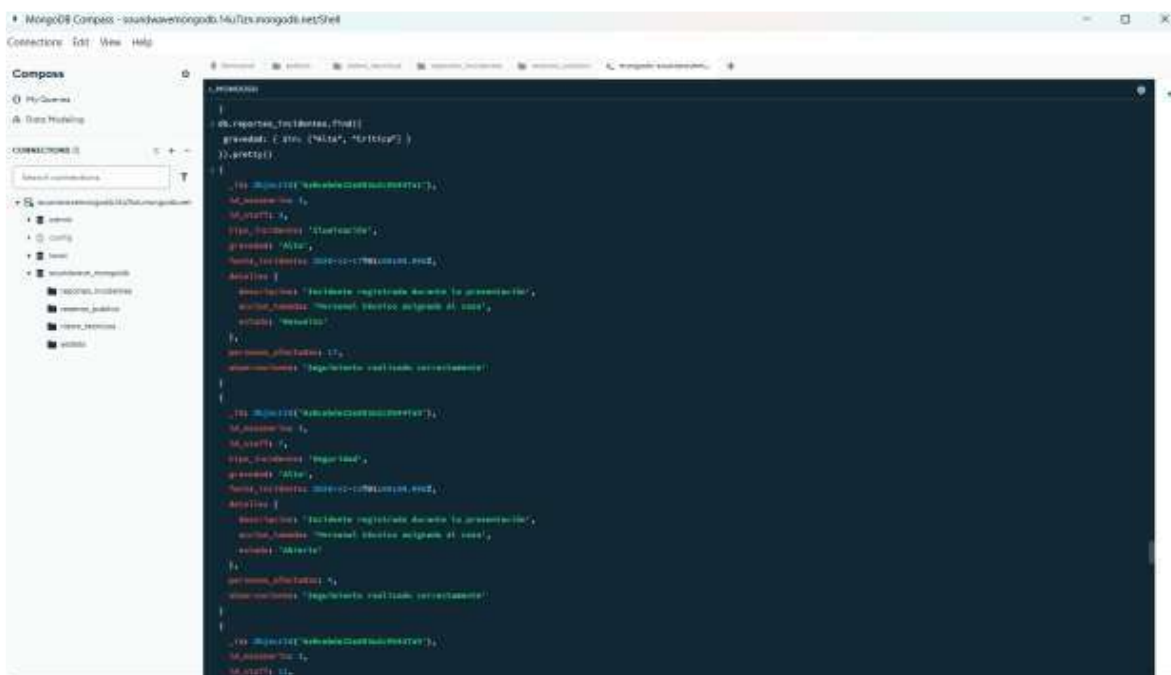


	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

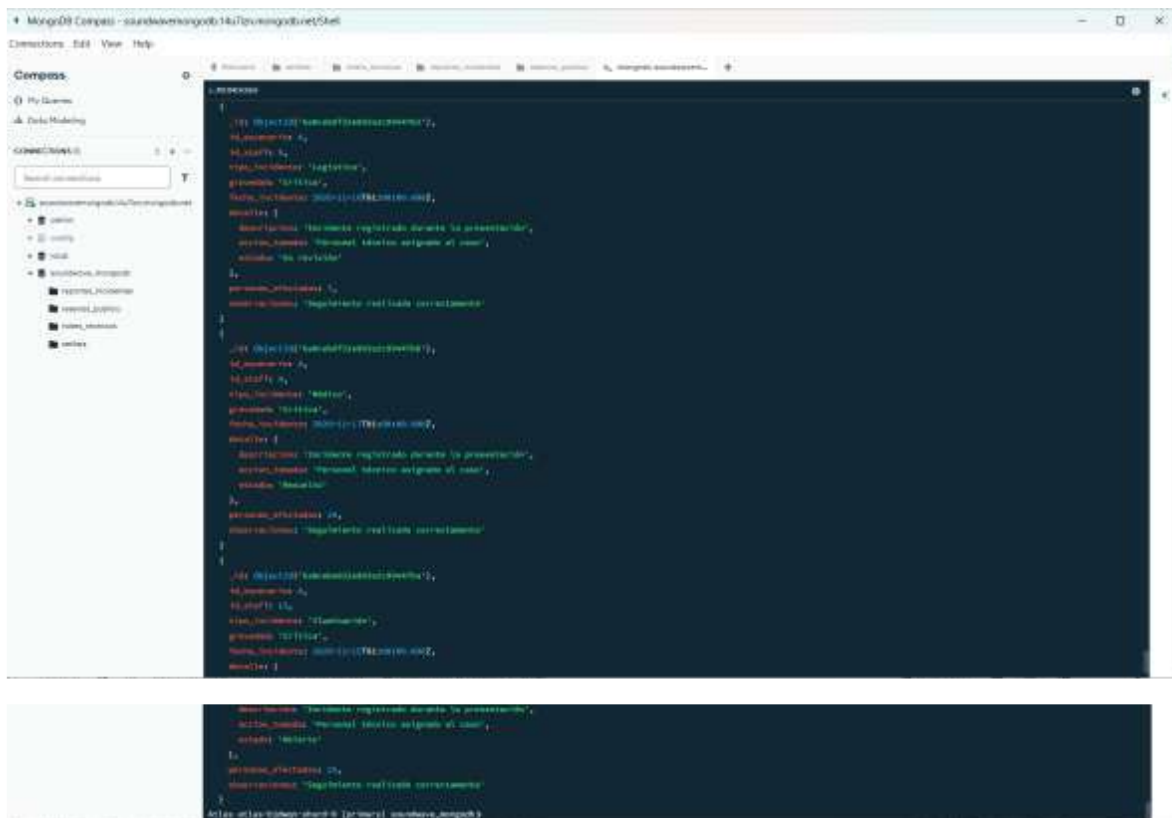
Consulta 2 - Incidentes graves con personas afectadas

Técnica: find() con operadores \$and, \$in y \$gte combinados para filtro múltiple. Propósito: Identificar los incidentes más críticos del festival que tuvieron personas afectadas y fueron resueltos, para análisis de gestión de riesgos.

```
db.reportes_incidentes.find({
  $and: [
    { gravedad: { $in: ["Alta", "Critica"] } },
    { personas_afectadas: { $gte: 1 } },
    { "detalle.estado": "Resuelto" }
  ]
}).sort({ personas_afectadas: -1 });
```



	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	



```

{
  "_id": "60a11111-1111-1111-1111-111111111111",
  "id_presentacion": "P001",
  "nombre": "Agenda",
  "descripcion": "Agenda",
  "escenario": "Escenario 1",
  "reviews": [
    {
      "id_usuario": "U001",
      "calificacion": 5,
      "comentario": "Muy buena la agenda, me gusta mucho."
    },
    {
      "id_usuario": "U002",
      "calificacion": 4,
      "comentario": "Buena la agenda, pero me gusta más la de la competencia."
    }
  ]
},
{
  "_id": "60a11111-1111-1111-1111-111111111111",
  "id_presentacion": "P002",
  "nombre": "Módulo",
  "descripcion": "Módulo",
  "escenario": "Escenario 2",
  "reviews": [
    {
      "id_usuario": "U003",
      "calificacion": 3,
      "comentario": "El módulo es bueno, pero me gusta más la de la competencia."
    },
    {
      "id_usuario": "U004",
      "calificacion": 4,
      "comentario": "Buena la agenda, pero me gusta más la de la competencia."
    }
  ]
},
{
  "_id": "60a11111-1111-1111-1111-111111111111",
  "id_presentacion": "P003",
  "nombre": "Escenario",
  "descripcion": "Escenario",
  "escenario": "Escenario 3",
  "reviews": [
    {
      "id_usuario": "U005",
      "calificacion": 5,
      "comentario": "Muy buena la agenda, me gusta mucho."
    },
    {
      "id_usuario": "U006",
      "calificacion": 4,
      "comentario": "Buena la agenda, pero me gusta más la de la competencia."
    }
  ]
}

```

Consulta 3 - Reporte de calidad por escenario (Aggregation Pipeline)

Técnica: Aggregation Pipeline con 6 etapas: \$lookup, \$unwind, \$group, \$addFields, \$sort y \$project. Propósito: Calcular métricas de satisfacción del público por escenario cruzando datos entre dos colecciones de MongoDB.

```

db.resenas_publico.aggregate([
  { $lookup: {
    from: "setlists",
    localField: "id_presentacion",
    foreignField: "id_presentacion",
    as: "info_presentacion"
  }},
  { $unwind: "$info_presentacion" },
  { $group: {
    _id: "$info_presentacion.escenario",

```

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

```

    promedio_calificacion: { $avg: "$calificacion" },
    total_resenas: { $sum: 1 },
    resenas_positivas: {
      $sum: { $cond: [{ $gte: ["$calificacion", 4] }, 1, 0] }
    },
    calificacion_maxima: { $max: "$calificacion" },
    calificacion_minima: { $min: "$calificacion" }
  }},
  { $addFields: {
    porcentaje_satisfaccion: {
      $round: [{ $multiply: [
        { $divide: ["$resenas_positivas", "$total_resenas"] }, 100
      ]}, 1]
    }
  }},
  { $sort: { promedio_calificacion: -1 } },
  { $project: {
    escenario: "$_id",
    promedio_calificacion: { $round: ["$promedio_calificacion", 2] },
    total_resenas: 1, resenas_positivas: 1,
    porcentaje_satisfaccion: 1,
    calificacion_maxima: 1, calificacion_minima: 1,
    _id: 0
  }}
]);

```

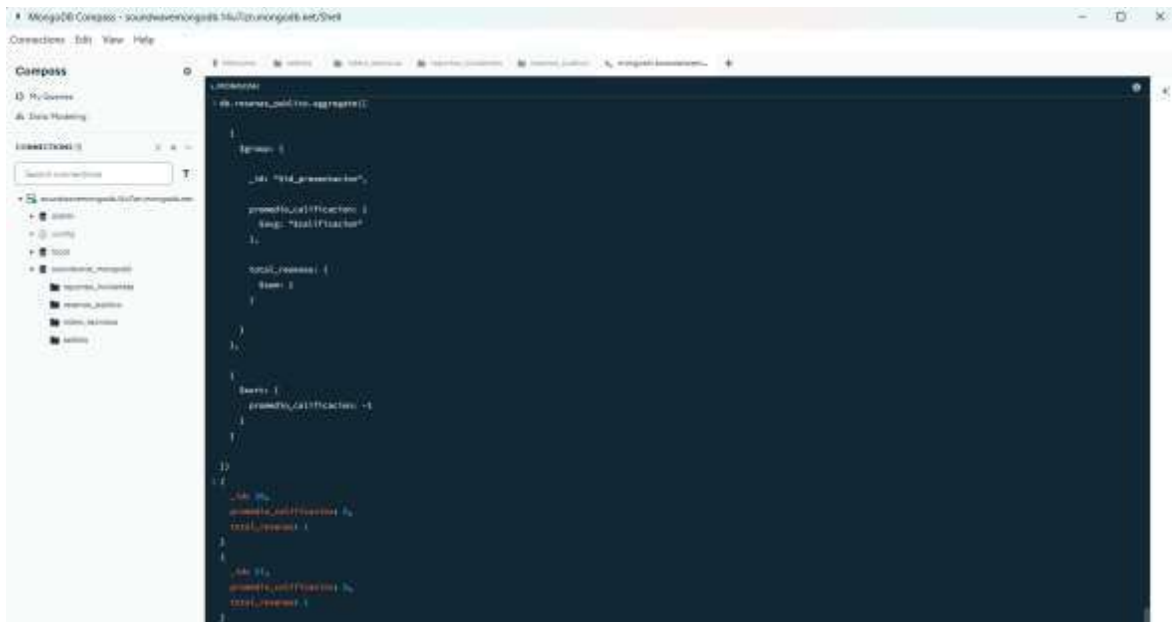
	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

Explicación etapa por etapa:

- \$lookup: cruza resenas_publico con setlists por id_presentacion para obtener el escenario de cada reseña
- \$unwind: desanida el array resultante del lookup
- \$group: agrupa por escenario calculando promedio, total, reseñas positivas, máximo y mínimo
- \$addFields: calcula el porcentaje de satisfacción como $\text{resenas_positivas} / \text{total} * 100$
- \$sort: ordena por promedio de mayor a menor
- \$project: formatea y renombra los campos del resultado final

Resultado obtenido:

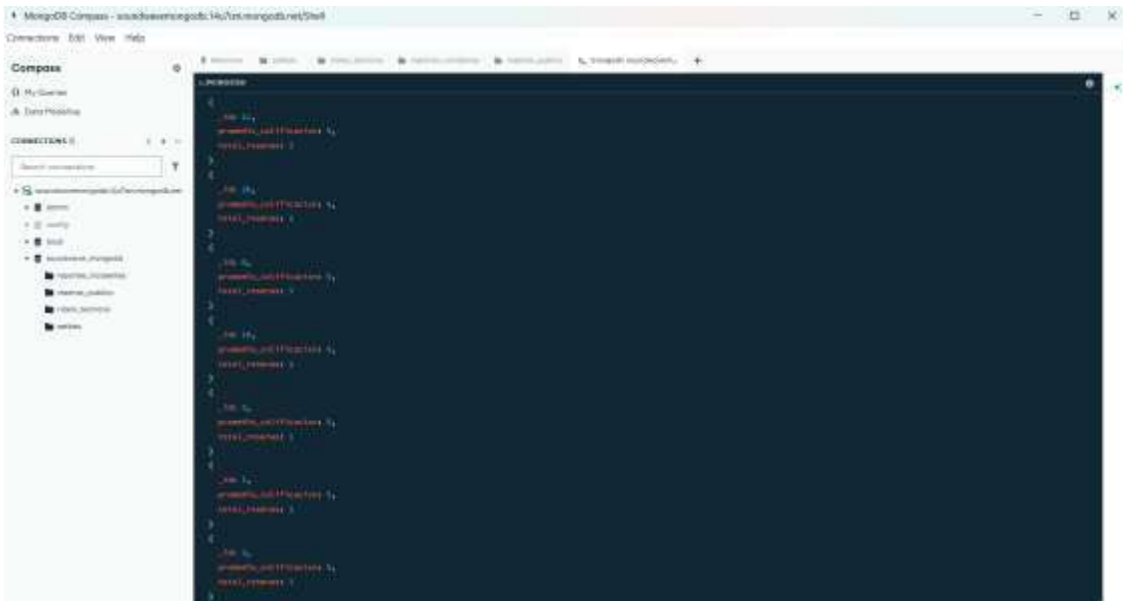
Escenario	Promedio	Reseñas	Satisfacción
Mainstage	4.5	8	87.5%
Nebula	4.25	8	87.5%
Underground	4.17	6	66.7%
Roots	3.75	8	75.0%



The screenshot shows the MongoDB Compass interface. On the left, the 'Compass' sidebar is visible with a search bar and a list of collections. The main area displays a query in the 'Query' tab, which is a JSON document. The query is as follows:

```
{
  "query": {
    "$lookup": {
      "from": "setlists",
      "localField": "id_presentacion",
      "foreignField": "id_presentacion",
      "as": "setlists"
    },
    "$unwind": {
      "path": "$setlists"
    },
    "$group": {
      "_id": "$setlists.escenario",
      "total_resenas": {
        "$sum": 1
      },
      "promedio_satisfaccion": {
        "$avg": "$satisfaccion"
      }
    },
    "$sort": {
      "promedio_satisfaccion": -1
    }
  },
  "projection": {
    "_id": 1,
    "total_resenas": 1,
    "promedio_satisfaccion": 1
  }
}
```

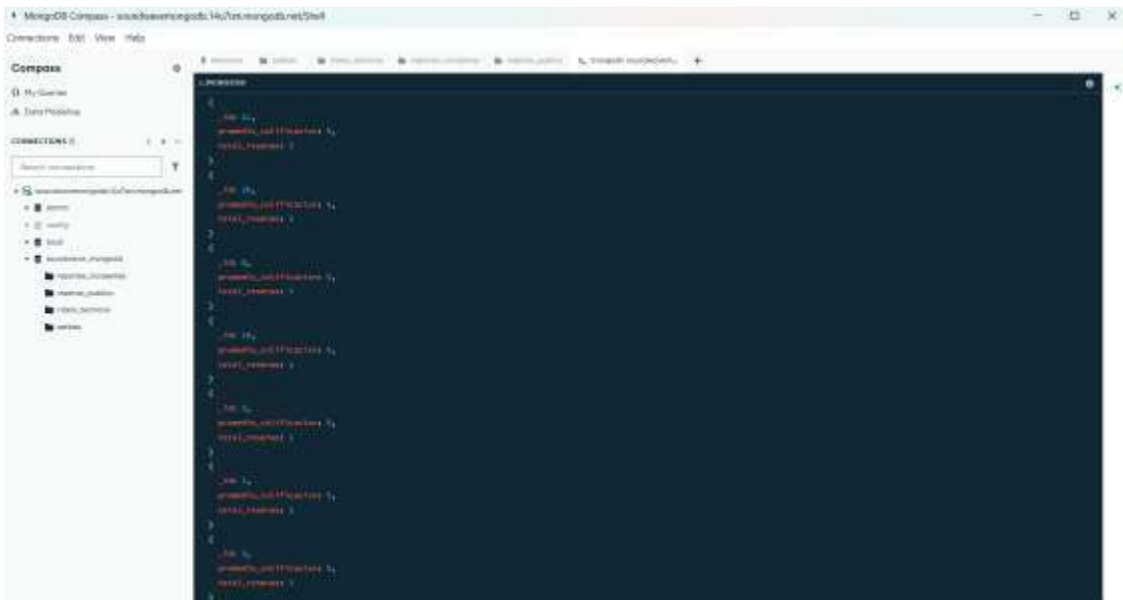
	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	



```

{ "_id": 1, "nombre_usuario": "jorge", "password": "123456", "email_usuario": "jorge@elbosque.edu.co" }
{ "_id": 2, "nombre_usuario": "maria", "password": "789012", "email_usuario": "maria@elbosque.edu.co" }
{ "_id": 3, "nombre_usuario": "carlos", "password": "345678", "email_usuario": "carlos@elbosque.edu.co" }
{ "_id": 4, "nombre_usuario": "ana", "password": "901234", "email_usuario": "ana@elbosque.edu.co" }
{ "_id": 5, "nombre_usuario": "pedro", "password": "567890", "email_usuario": "pedro@elbosque.edu.co" }
{ "_id": 6, "nombre_usuario": "luis", "password": "234567", "email_usuario": "luis@elbosque.edu.co" }
{ "_id": 7, "nombre_usuario": "maria", "password": "890123", "email_usuario": "maria@elbosque.edu.co" }
{ "_id": 8, "nombre_usuario": "carlos", "password": "456789", "email_usuario": "carlos@elbosque.edu.co" }
{ "_id": 9, "nombre_usuario": "ana", "password": "012345", "email_usuario": "ana@elbosque.edu.co" }
{ "_id": 10, "nombre_usuario": "pedro", "password": "678901", "email_usuario": "pedro@elbosque.edu.co" }

```



```

{ "_id": 1, "nombre_usuario": "jorge", "password": "123456", "email_usuario": "jorge@elbosque.edu.co" }
{ "_id": 2, "nombre_usuario": "maria", "password": "789012", "email_usuario": "maria@elbosque.edu.co" }
{ "_id": 3, "nombre_usuario": "carlos", "password": "345678", "email_usuario": "carlos@elbosque.edu.co" }
{ "_id": 4, "nombre_usuario": "ana", "password": "901234", "email_usuario": "ana@elbosque.edu.co" }
{ "_id": 5, "nombre_usuario": "pedro", "password": "567890", "email_usuario": "pedro@elbosque.edu.co" }
{ "_id": 6, "nombre_usuario": "luis", "password": "234567", "email_usuario": "luis@elbosque.edu.co" }
{ "_id": 7, "nombre_usuario": "maria", "password": "890123", "email_usuario": "maria@elbosque.edu.co" }
{ "_id": 8, "nombre_usuario": "carlos", "password": "456789", "email_usuario": "carlos@elbosque.edu.co" }
{ "_id": 9, "nombre_usuario": "ana", "password": "012345", "email_usuario": "ana@elbosque.edu.co" }
{ "_id": 10, "nombre_usuario": "pedro", "password": "678901", "email_usuario": "pedro@elbosque.edu.co" }

```



```

{ "_id": 1, "nombre_usuario": "jorge", "password": "123456", "email_usuario": "jorge@elbosque.edu.co" }
{ "_id": 2, "nombre_usuario": "maria", "password": "789012", "email_usuario": "maria@elbosque.edu.co" }
{ "_id": 3, "nombre_usuario": "carlos", "password": "345678", "email_usuario": "carlos@elbosque.edu.co" }
{ "_id": 4, "nombre_usuario": "ana", "password": "901234", "email_usuario": "ana@elbosque.edu.co" }

```

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

5. ENTREGA FINAL - INTEGRACIÓN POLÍGLOTA

5.1. Script de integración

Archivo: script_integracion.py **Lenguaje:** Python 3

Dependencias:

Librería	Uso
psycopg2-binary	Conector PostgreSQL
pymongo	Conector MongoDB

5.2. Caso 1 - Reporte de Artista

Combina en un solo panel los datos del artista y su contrato desde PostgreSQL con el setlist completo desde MongoDB.

Datos desde PostgreSQL:

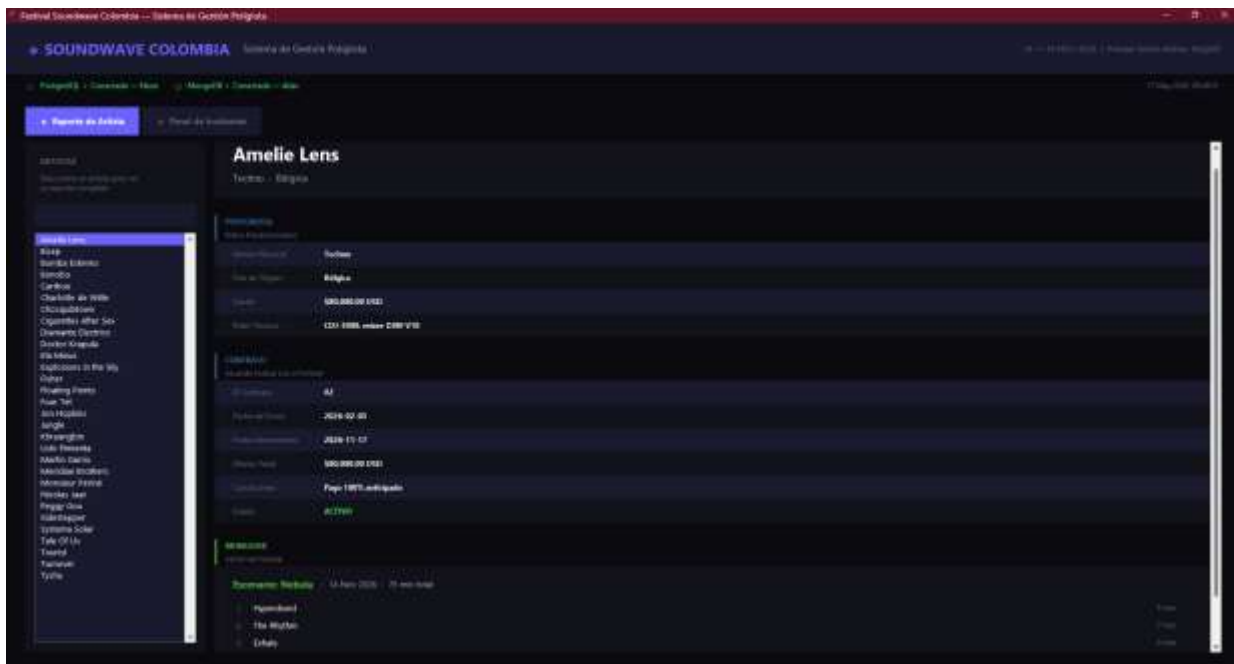
- Nombre, género musical, país de origen y caché del artista
- ID de contrato, fecha de firma, fecha de vencimiento, monto total, condiciones y estado

Datos desde MongoDB:

- Escenario y fecha de presentación
- Lista completa de canciones con orden y duración individual
- Duración total del set en minutos

Funcionamiento: El usuario selecciona un artista de la lista con buscador en tiempo real. El sistema consulta simultáneamente PostgreSQL y MongoDB y presenta toda la información integrada en un panel unificado con scroll.

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	



5.3. Caso 2 - Panel de Incidentes por Escenario

Combina en un solo panel los datos del escenario y su programación desde PostgreSQL con los reportes de incidentes y reseñas desde MongoDB.

Datos desde PostgreSQL:

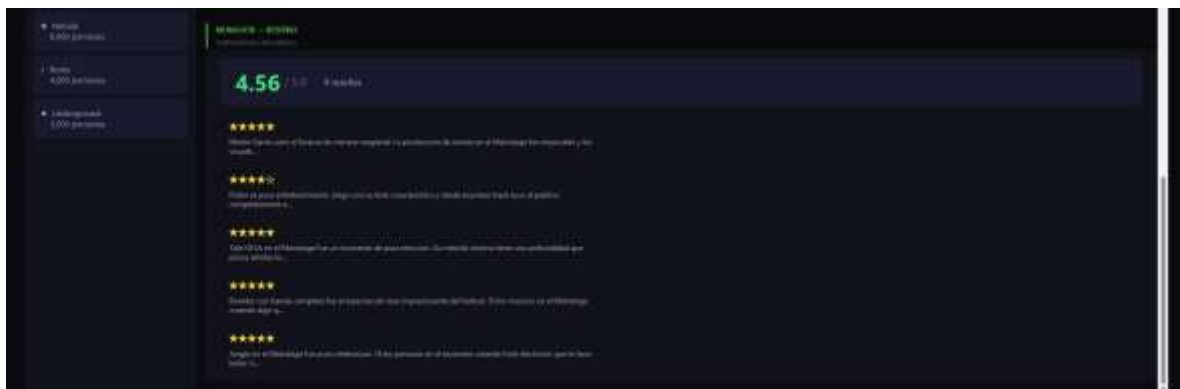
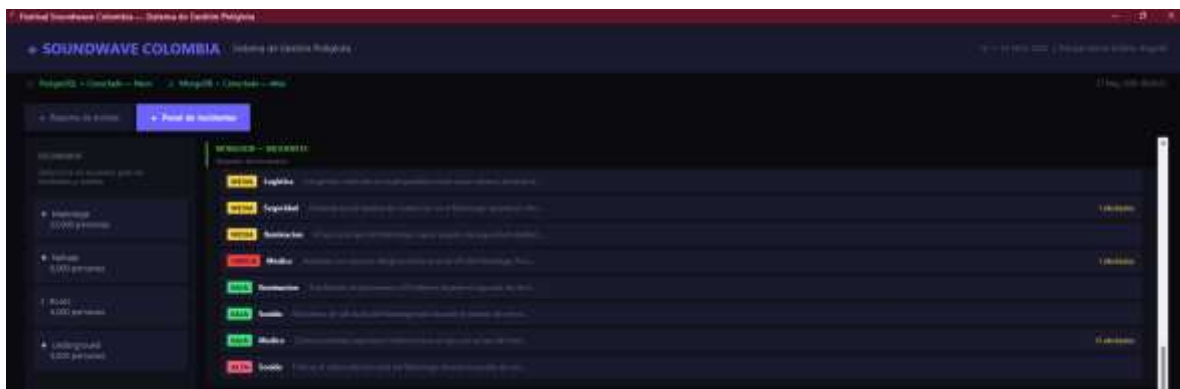
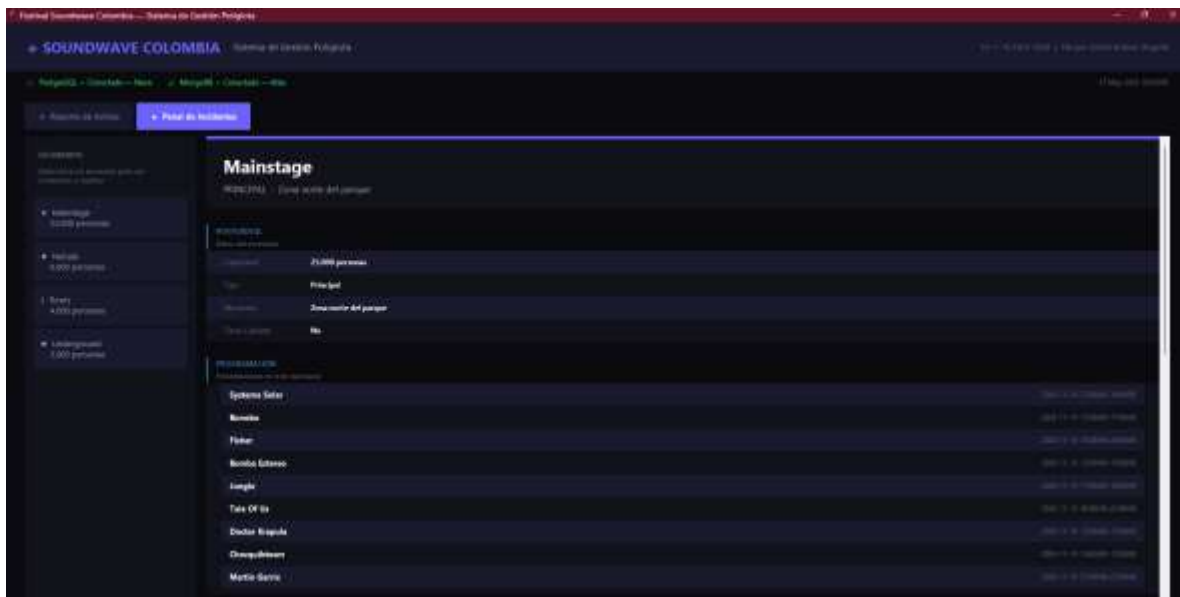
- Nombre, capacidad, tipo y ubicación del escenario
- Programación completa de presentaciones con artista, fecha y horario

Datos desde MongoDB:

- Lista de incidentes con badge de gravedad por nivel: Baja, Media, Alta, Critica
- Promedio de calificaciones con número grande y barra visual de progreso
- Total de reseñas recibidas
- Las 5 reseñas más recientes con estrellas y comentario completo

Funcionamiento: El usuario selecciona uno de los 4 escenarios del panel izquierdo. El sistema consulta PostgreSQL y MongoDB y presenta toda la información integrada en un panel unificado con scroll.

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	



	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

6. JUSTIFICACIÓN TEÓRICA — ACID, BASE Y CAP

6.1. Propiedades ACID en PostgreSQL

PostgreSQL implementa las cuatro propiedades ACID en el Festival Soundwave Colombia. Cada una es necesaria por razones concretas del negocio:

Atomicidad

Una transacción se ejecuta completa o no se ejecuta. En el festival esto es crítico en la venta de boletas: cuando un asistente compra una boleta, el sistema debe registrar la venta en la tabla Ventas y verificar el cupo en Tipos_Boleta como una sola unidad indivisible. Si el sistema falla a mitad de la operación, no puede quedar una venta registrada sin que el cupo se haya actualizado, ni un cupo decrementado sin venta registrada. Ambas operaciones ocurren juntas o ninguna ocurre.

Consistencia

La base de datos siempre pasa de un estado válido a otro estado válido. En el festival esto se garantiza mediante las restricciones CHECK, UNIQUE y FOREIGN KEY definidas en el esquema. No puede existir una presentación con una fecha fuera del rango del festival, no puede existir una venta que referencie un asistente inexistente, y no puede existir un contrato que referencie un artista no registrado. El trigger `trg_validar_aforo` también hace parte de esta garantía al verificar que el cupo nunca se supere.

Aislamiento

Las transacciones concurrentes no se interfieren entre sí. En el festival esto es especialmente relevante cuando múltiples personas compran boletas del mismo tipo simultáneamente. El trigger `trg_validar_aforo` usa FOR UPDATE para bloquear la fila del tipo de boleta durante la verificación del cupo, garantizando que dos compras simultáneas no puedan superar el límite al mismo tiempo.

Durabilidad

Una vez confirmada una transacción, los datos persisten, aunque el sistema falle. En el festival esto garantiza que una venta confirmada no se pierda, aunque Neon sufra una interrupción. El asistente que compró su boleta VIP tiene la garantía de que su transacción quedó registrada permanentemente.

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

6.2 Consistencia eventual (BASE) en MongoDB

MongoDB opera bajo el modelo BASE en el Festival Soundwave Colombia. A diferencia de PostgreSQL donde cada operación debe ser inmediatamente consistente, MongoDB acepta que el estado de la base de datos converja hacia la consistencia con un pequeño retraso.

La distinción fundamental entre las colecciones transaccionales de PostgreSQL y las documentales de MongoDB es el impacto económico y de seguridad de una inconsistencia temporal. Cuando se vende una boleta VIP en PostgreSQL, una inconsistencia de un segundo podría resultar en dos personas con el mismo asiento, lo cual tiene consecuencias reales e inmediatas. En cambio, cuando un asistente publica una reseña en MongoDB, que esa reseña tarde 30 segundos en aparecer en el ranking de presentaciones no afecta a nadie ni genera ningún perjuicio.

Las cuatro colecciones MongoDB del festival comparten esta característica: su información es descriptiva y posterior al evento, no prescriptiva ni transaccional. Los setlists se definen días antes y su actualización de último minuto no bloquea ningún proceso crítico. Los riders técnicos son documentos de planificación que se consultan por el equipo técnico, no por el sistema de ventas. Las reseñas son opiniones que se generan después de las presentaciones y cuya visibilidad retrasada no afecta la experiencia de ningún otro asistente. Los reportes de incidentes son registros operativos que se documentan para análisis posterior, no para toma de decisiones en tiempo real donde una fracción de segundo marque la diferencia.

6.3 Posición en el triángulo CAP

El teorema CAP establece que un sistema distribuido solo puede garantizar dos de tres propiedades simultáneamente: Consistencia (C), Disponibilidad (A) y Tolerancia a particiones (P).

PostgreSQL en Neon - Sistema CP

PostgreSQL prioriza Consistencia y Tolerancia a particiones, sacrificando Disponibilidad en casos extremos.

Propiedad	Se garantiza	Aplicación en el festival
Consistencia	Si	Todos los nodos ven el mismo estado de ventas y cupos
Disponibilidad	Parcial	En una partición de red el sistema puede rechazar escrituras para mantener consistencia
Tolerancia a particiones	Si	El sistema tolera fallos de red sin perder integridad

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

Lo que se sacrifica conscientemente: En el improbable caso de una partición de red, PostgreSQL puede volverse temporalmente no disponible para escrituras antes de comprometer la consistencia de los datos transaccionales. Para el festival esto es aceptable: es preferible que el sistema de ventas esté temporalmente caído a que se vendan más boletas de las permitidas o que dos personas reciban la misma boleto VIP.

MongoDB Atlas - Sistema AP

MongoDB prioriza Disponibilidad y Tolerancia a particiones, sacrificando Consistencia estricta.

Propiedad	Se garantiza	Aplicación en el festival
Consistencia	Eventual	Las reseñas e incidentes pueden tener ligero retraso en propagarse
Disponibilidad	Si	El sistema siempre responde, aunque haya partición de red
Tolerancia a particiones	Si	El cluster continúa operando ante fallos de nodos individuales

Lo que se sacrifica conscientemente: La consistencia inmediata entre nodos. Para el festival esto es aceptable porque las colecciones en MongoDB no manejan dinero, cupos ni contratos. Una reseña que llega con segundos de retraso o un reporte de incidente que no es instantáneamente visible en todos los nodos no representa ningún riesgo económico ni de seguridad para el evento.

Resumen de la arquitectura CAP del sistema:

Motor	Tipo CAP	Sacrifica	Justificación
PostgreSQL (Neon)	CP	Disponibilidad parcial	Las ventas y contratos requieren consistencia estricta por encima de todo
MongoDB (Atlas)	AP	Consistencia inmediata	Las reseñas, setlists e incidentes toleran consistencia eventual sin riesgo

	Programa de Ingeniería de Sistemas	Informe Final
	Bases de Datos 2	

7. REPOSITORIO GITHUB

El código fuente completo del proyecto está disponible en el repositorio público:

<https://github.com/stiusaba/festival-bd2-Festival8.git>